KPR Institute of Engineering and Technology

Department of Artificial Intelligence and Data Science

U21ADP05 - Exploratory Data Analysis and Visualization

Assignment II - Case Study

PROJECT REPORT

Exploratory Data Analysis, Visualization and Deep Learning Based Churn Prediction on the Telco Customer Dataset

Student Name	Prashant Sharma Dhungana
Roll No.	24AD130
Department	Artificial Intelligence and Data Science (Section AD-B)
Year / Semester	III Year / V Semester (Batch 2024-2028)
Course Faculty	Ms Revathy M
Date of Submission	15/10/2026
GitHub Repository	github.com/Its-PrashantTechie/Telco-Customer-Churn-Prediction

Abstract

Customer churn is a major source of revenue loss for telecom companies. This project applies exploratory data analysis, statistical testing and visualization to the IBM Telco Customer Churn dataset (7,043 customers, 21 fields) to find why customers leave and to predict who is likely to leave. Data cleaning, encoding, scaling and a stratified 70/15/15 split were performed, followed by eight visualizations and distribution and hypothesis tests. A Multi-Layer Perceptron was tuned over twelve hyper-parameter settings. Churn (26.5%) is strongly linked to short tenure, month-to-month contracts, fibre-optic service and electronic-check payment, while gender is irrelevant. On unseen data the model achieved an AUC of 0.839, accuracy of 74.6% and recall of 76.2%, matching a logistic regression baseline.

Keywords: Exploratory data analysis, data visualization, customer churn, deep learning, MLP, statistical analysis.

1. Introduction and Objective

Telecom is a highly competitive industry in which acquiring a new customer costs several times more than retaining an existing one. A company that can understand the reasons behind churn, and identify at-risk customers early, can run targeted retention offers and protect revenue. Exploratory Data Analysis (EDA) helps discover patterns before modelling, and a deep learning model can then turn those patterns into a predictive tool.

Objectives of this case study:

- Explore, clean and understand a real-world customer dataset with more than 15 fields.
- Create meaningful visualizations covering all seven stages of visualization (acquire, parse, filter, mine, represent, refine, interact).
- Study continuous and discrete distributions and test relationships statistically.
- Build and tune a deep learning model (MLP) to predict churn and evaluate it with loss/accuracy curves, confusion matrix and ROC-AUC.
- Draw practical conclusions and recommendations for the business.

2. Dataset Description

Source: IBM sample "Telco Customer Churn" dataset, downloaded from Kaggle (kaggle.com/datasets/blastchar/telco-customer-churn) [1].

Size: 7,043 customer records and 21 fields (3 numeric, 17 categorical including the target). Target variable: Churn (Yes = customer left within the last month). Churn rate = 26.5% (1,869 churned, 5,174 retained), so the data is imbalanced.

Group	Fields
Customer demographics	gender, SeniorCitizen, Partner, Dependents
Account information	tenure, Contract, PaperlessBilling, PaymentMethod,

	MonthlyCharges, TotalCharges
Services subscribed	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
Identifier / Target	customerID (dropped), Churn (target)

Numeric feature	Mean	Std	Min	Median	Max	Skewness
tenure	32.37	24.56	0.0	29.0	72.0	0.24
MonthlyCharges	64.76	30.09	18.25	70.35	118.75	-0.22
TotalCharges	2279.73	2266.79	0.0	1394.55	8684.8	0.96

Table 1: Basic statistics of the numeric features.

3. EDA and Preprocessing

3.1 Methods used

- **Duplicates:** checked on complete rows and on customerID - none found.
- **Missing values:** TotalCharges was stored as text because 11 cells were blank. All 11 customers have tenure = 0 (new customers not yet billed), so the blank was replaced by 0 instead of a mean/median.
- **Outliers:** detected with the 1.5 x IQR rule on tenure, MonthlyCharges and TotalCharges. No value fell outside the fences and all values are realistic, so nothing was removed.
- **Transformation / encoding:** customerID dropped; SeniorCitizen converted to Yes/No; 6 binary columns label-encoded (0/1); 10 multi-category columns one-hot encoded, giving 40 input features.
- **Scaling:** tenure, MonthlyCharges and TotalCharges standardised (z-score). The scaler was fitted on the training set only to prevent data leakage.
- **Splitting:** stratified split into train 4,930 (70%), validation 1,056 (15%) and test 1,057 (15%); the churn ratio is 26.5% in every part.

3.2 Insights

The data is of high quality (only one hidden quality issue). The main modelling concerns are class imbalance (handled with class weights and recall/F1/AUC metrics) and the strong correlation between tenure and TotalCharges (r = 0.83), since total charges is roughly monthly charges multiplied by months.

4. Data Visualization

All seven stages of visualization were followed: Acquire (downloaded CSV), Parse (types fixed), Filter (cleaning, outlier check), Mine (statistical tests), Represent (charts below), Refine (labels,

colour coding, annotations) and Interact (Plotly chart supplied as figures/08_interactive_scatter.html). Python (Matplotlib, Seaborn, Plotly) was used.

4.1 Target distribution and churn by contract (bar charts)

What it shows: Number of churned vs retained customers and the churn rate for each contract type.

Why created: To understand class balance, which affects modelling, and to test the most obvious business driver.

Insights: 26.5% of customers churn. Churn is 42.7% for month-to-month contracts, 11.3% for one-year and only 2.8% for two-year contracts.

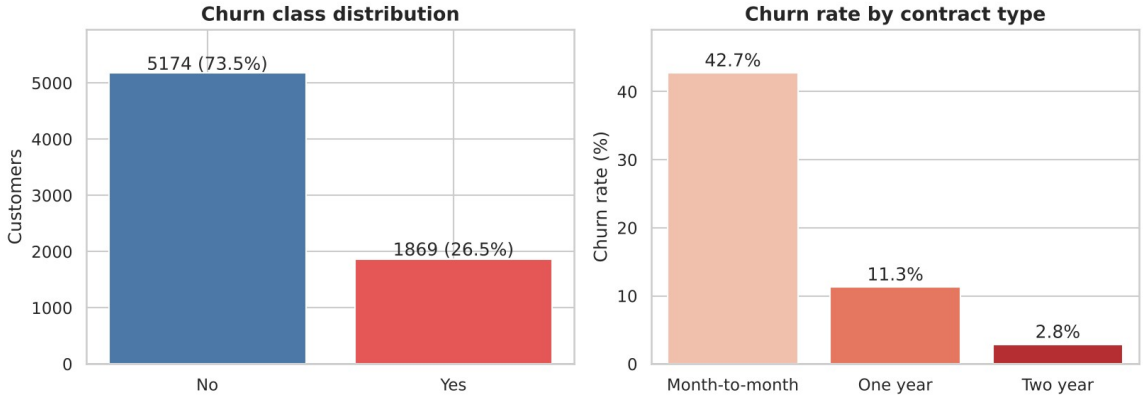


Figure 1: Class distribution and churn rate by contract type

4.2 Histograms with KDE split by churn

What it shows: Distribution of tenure, MonthlyCharges and TotalCharges for churners and non-churners.

Why created: To see the shape of each numeric variable and where the two groups differ.

Insights: Churners are concentrated at very low tenure (mean 18.0 vs 37.6 months) and pay more per month (mean $74.4 vs $61.3). Non-churners show a large spike at 72 months (long-term loyal customers) and at about $20 monthly (basic phone-only plan).

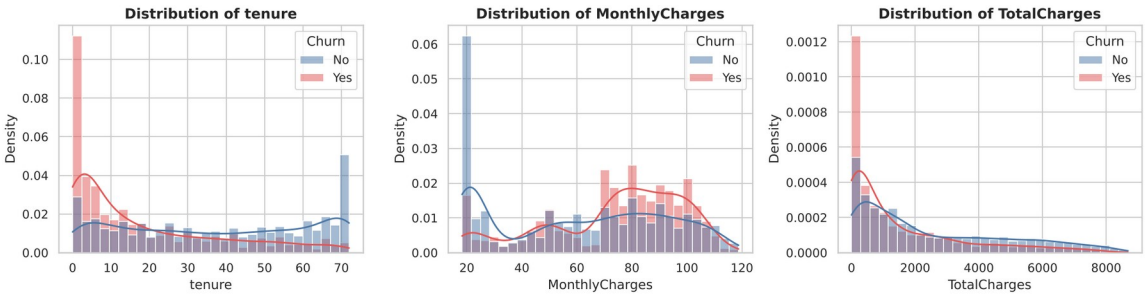


Figure 2: Histograms with KDE of the numeric features by churn

4.3 Boxplots by churn

What it shows: Median, spread and outliers of each numeric variable for the two classes.

Why created: Boxplots compare groups compactly and reveal outliers.

Insights: The median tenure of churners is about 10 months versus 38 for retained customers; churners have a higher median monthly bill (about $80 vs $64). The dots in the TotalCharges plot are the long right tail of the churn group, not data errors.

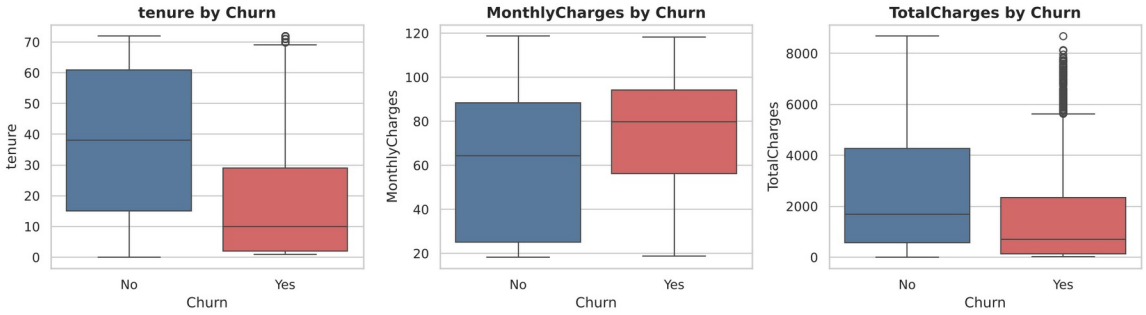


Figure 3: Boxplots of numeric features by churn

4.4 Correlation heatmap

What it shows: Pearson correlation between numeric features, the number of add-on services and churn.

Why created: To spot relationships and multicollinearity before modelling.

Insights: Tenure has the strongest correlation with churn (-0.35), followed by TotalCharges (-0.20) and MonthlyCharges (+0.19). Tenure and TotalCharges are highly correlated (0.83), so they carry overlapping information.

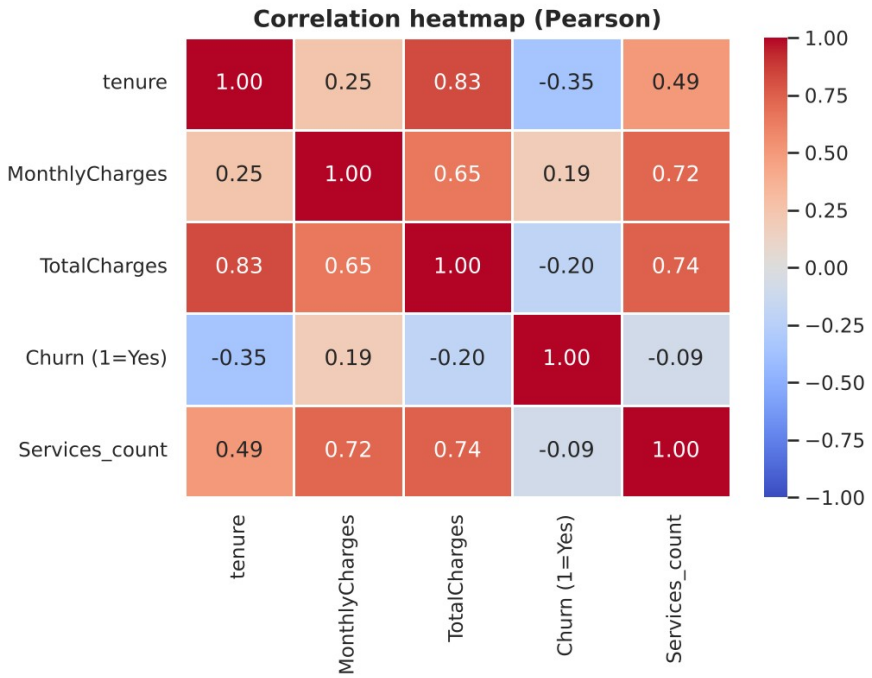


Figure 4: Correlation heatmap

4.5 Scatter plot: tenure vs monthly charges

What it shows: Each customer as a point coloured by churn status.

Why created: To view two variables together and locate the "risky" region.

Insights: Churners (red) cluster in the top-left: new customers on high monthly plans ($70-$105). The long-tenure region is almost entirely retained customers.

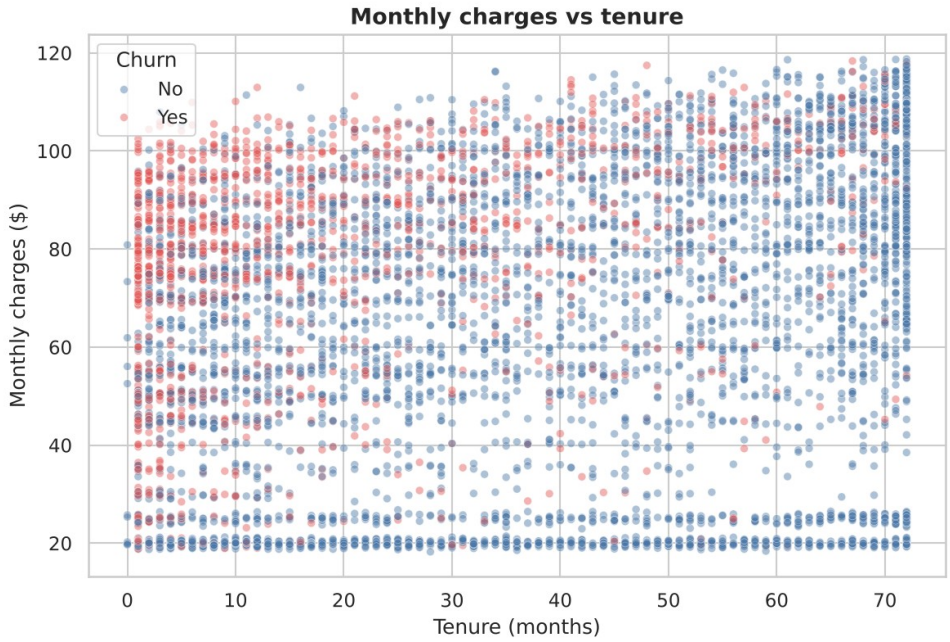


Figure 5: Monthly charges vs tenure

4.6 Pairplot

What it shows: All pairwise scatter plots and marginal distributions of the numeric features by churn class.

Why created: To get a multivariate overview in one figure.

Insights: It confirms the near-linear relation between tenure and TotalCharges and shows that churners are separated mainly along the tenure axis.

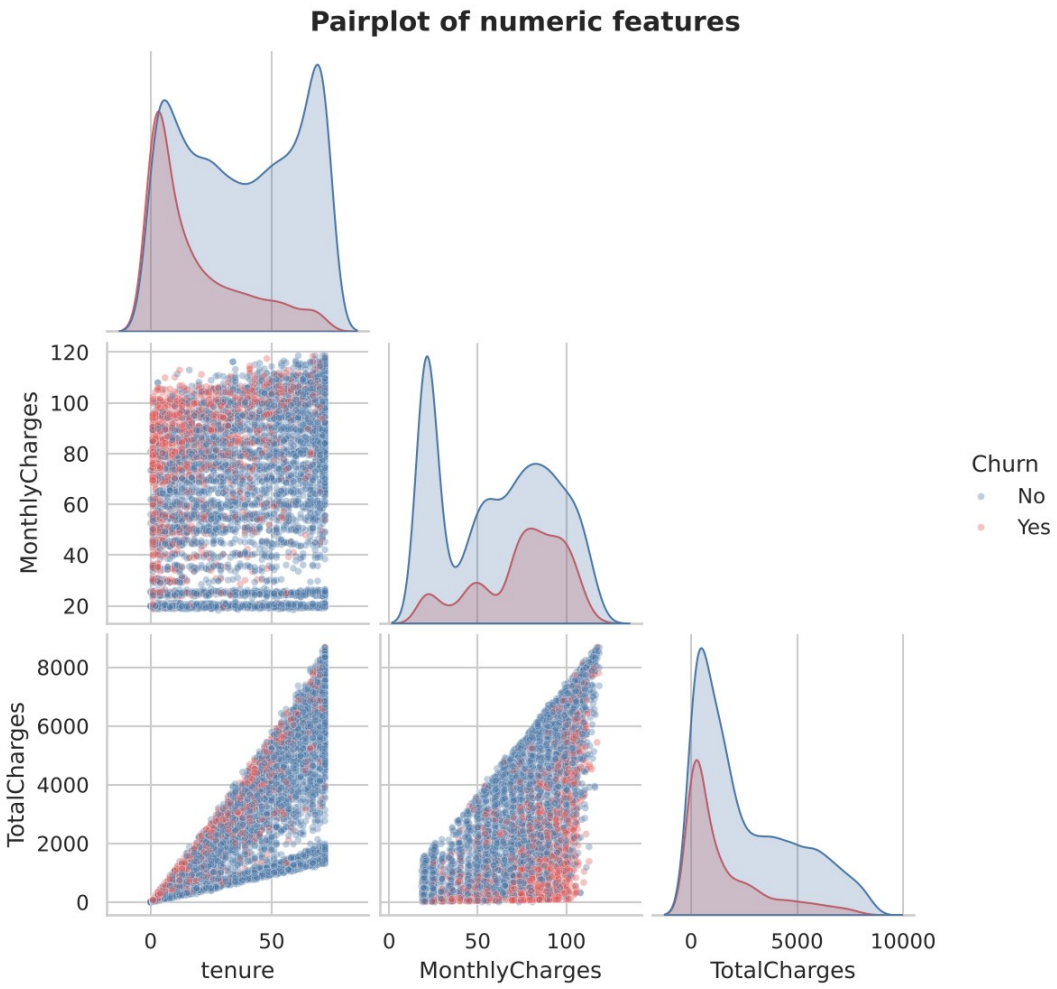


Figure 6: Pairplot of numeric features

4.7 Churn rate for every category level

What it shows: The churn rate of each level of every categorical feature, compared with the overall rate of 26.5% (dashed line).

Why created: To quickly identify which customer segments are high-risk.

Insights: Highest-risk segments: electronic-check payers (45.3%), month-to-month contracts (42.7%), fibre-optic internet (41.9%), no Online Security (41.8%) and senior citizens (41.7%). Safest: two-year contracts (2.8%) and customers with no internet service (7.4%).

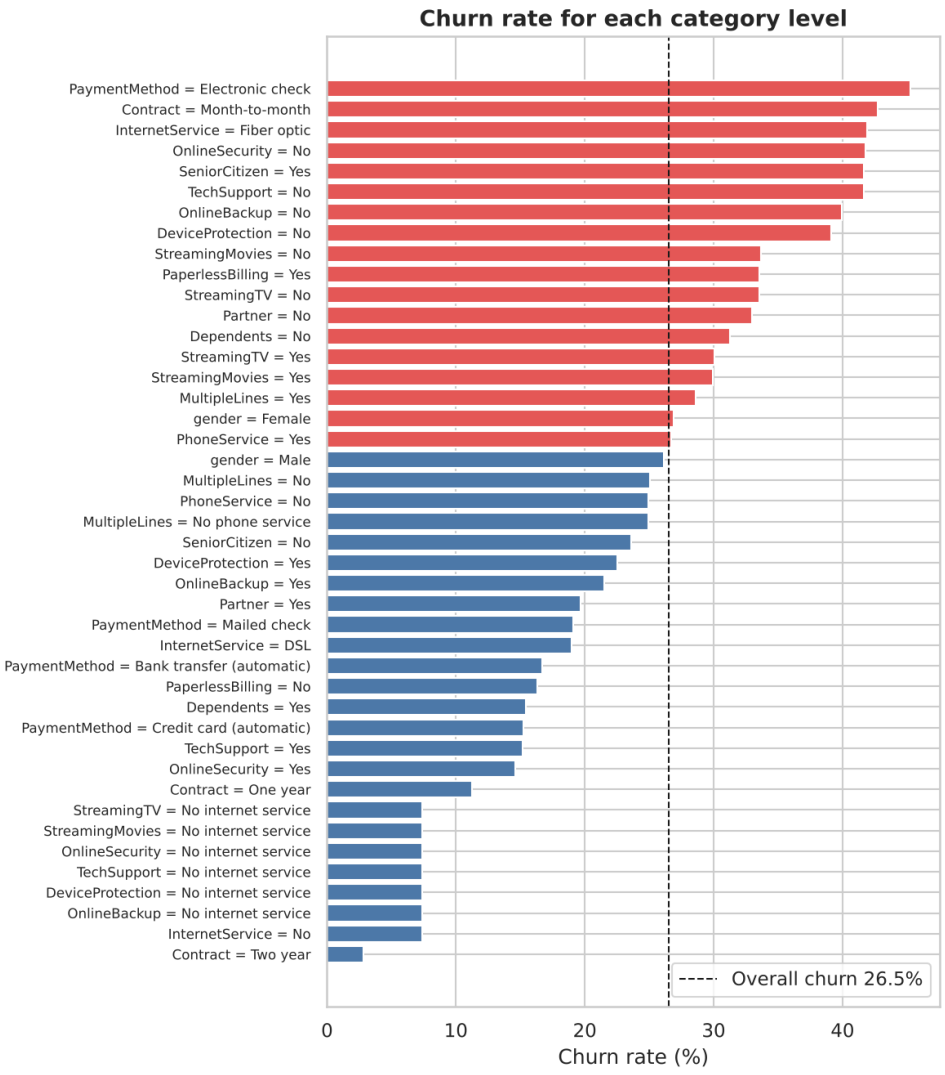


Figure 7: Churn rate by category level

4.8 Interactive scatter chart (Stage 7: Interact)

What it shows: The tenure vs monthly charges scatter plot as an interactive Plotly page with hover details, zoom and legend filtering; marker shape indicates contract type.

Why created: To let a business user explore individual customer segments dynamically.

Insights: Provided as figures/08_interactive_scatter.html in the repository (open in any browser).

5. Statistical and Distribution Analysis

5.1 Continuous distributions

Each numeric feature was compared with a fitted Normal curve and a Q-Q plot, and tested with the D'Agostino-Pearson normality test.

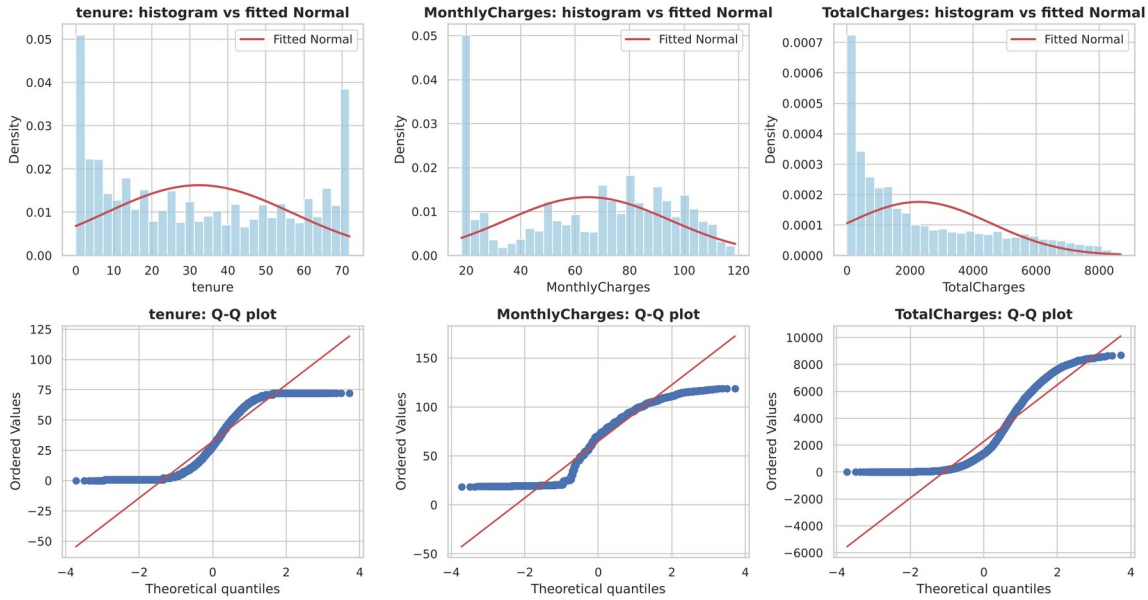


Figure 8: Histograms with fitted Normal curve and Q-Q plots

Feature	Skewness	Excess kurtosis	K2 statistic	p-value	Normal?
tenure	0.24	-1.387	76,258	< 0.001	No
MonthlyCharges	-0.221	-1.257	11,420	< 0.001	No
TotalCharges	0.963	-0.229	809	< 0.001	No

All three variables reject normality. Tenure and MonthlyCharges are flat/multi-peaked (excess kurtosis about -1.3) and TotalCharges is right-skewed (skew 0.96). Therefore non-parametric tests (Mann-Whitney U) were used and features were standardised before the neural network.

5.2 Discrete distributions

- **Churn:** Churn follows a Bernoulli distribution with p = 0.265 (variance p(1-p) = 0.195).
- **Add-on services:** the number of add-on services (0-6) was compared with a Binomial(6, 0.34) model. A chi-square goodness-of-fit test rejects the Binomial ($p < 0.001$): there are far more customers with zero services than expected (31%), mainly because 1,526 customers have no internet and cannot buy add-ons.
- **Churn vs services:** Churn is highest for customers with a single add-on (45.8%) and falls steadily to 5.3% for customers with all six, so each extra service makes a customer stickier.

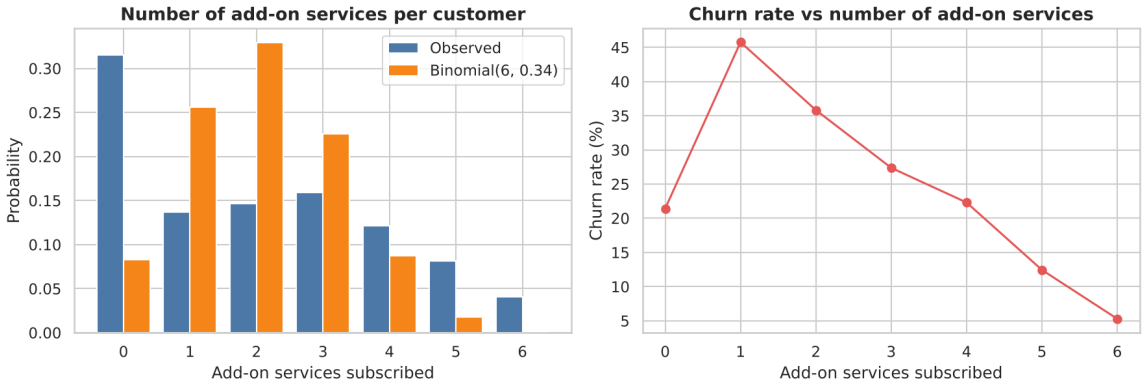


Figure 9: Observed vs Binomial distribution of add-on services, and churn rate by number of services

5.3 Bivariate analysis

Numeric features were compared between churners and non-churners with the Mann-Whitney U test; categorical features with the chi-square test of independence, using Cramer's V as the effect size.

Feature	Test	Effect size	p-value
Contract	Chi-square	0.41	5.86e-258
tenure	Mann-Whitney U	-0.352	2.42e-208
OnlineSecurity	Chi-square	0.347	2.66e-185
TechSupport	Chi-square	0.343	1.44e-180
InternetService	Chi-square	0.322	9.57e-160
PaymentMethod	Chi-square	0.303	3.68e-140
OnlineBackup	Chi-square	0.292	2.08e-131
DeviceProtection	Chi-square	0.282	5.51e-122

17 of the 19 features are significantly related to churn. Gender ($p = 0.49$) and PhoneService ($p = 0.34$) are not significant. Churn is therefore driven by contract commitment, tenure, support/security add-ons, internet type and payment method rather than by demographics.

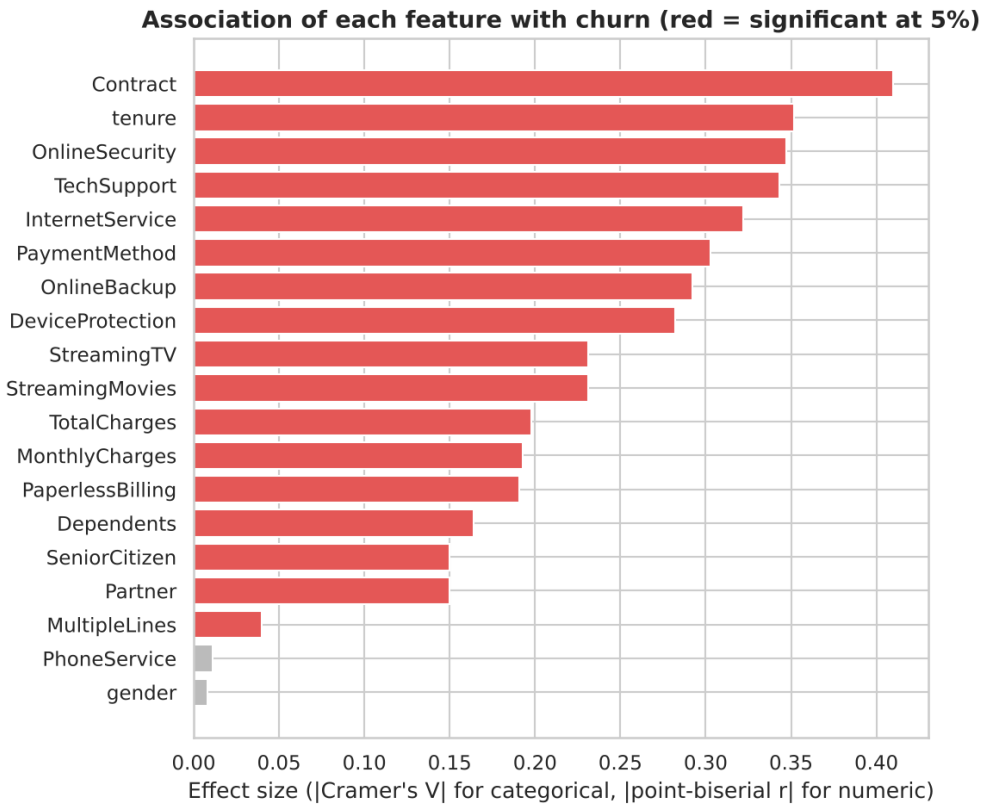


Figure 10: Strength of association of each feature with churn

6. Deep Learning Model

6.1 Architecture

A Multi-Layer Perceptron (feed-forward neural network) was chosen because the data is tabular. Architecture: Input(40) -> Dense(32, ReLU) -> Dropout(0.4) -> Dense(16, ReLU) -> Dropout(0.4) -> Dense(1, Sigmoid). The network has 1,857 trainable parameters, which is suitable for about 5,000 training rows.

6.2 Training parameters

Parameter	Value
Optimizer	Adam
Loss function	Binary cross-entropy
Batch size	64
Maximum epochs	100 (early stopping, patience 12 on validation AUC, best weights restored)
Epochs actually trained	22
Class imbalance	Balanced class weights (0: 0.68, 1: 1.88)
Weight initialiser	He normal
Random seed	42

6.3 Hyper-parameter tuning

A grid search over 12 combinations was done: hidden layer sizes {(32,16), (64,32), (128,64)} x dropout {0.2, 0.4} x learning rate {0.001, 0.01}. Configurations were ranked by validation AUC (the test set was never used for tuning).

Hidden units	Dropout	Learning rate	Epochs run	Validation AUC
(32, 16)	0.4	0.01	22	0.8459
(128, 64)	0.2	0.001	20	0.8457
(128, 64)	0.4	0.01	13	0.8456
(128, 64)	0.4	0.001	26	0.8454
(128, 64)	0.2	0.01	13	0.8443
(64, 32)	0.2	0.001	22	0.8433
(32, 16)	0.2	0.01	18	0.843
(64, 32)	0.2	0.01	15	0.8424
(64, 32)	0.4	0.01	16	0.8424
(32, 16)	0.4	0.001	41	0.8417
(64, 32)	0.4	0.001	19	0.8417
(32, 16)	0.2	0.001	30	0.8412

All configurations fall within a narrow band (0.841-0.846), so the model is insensitive to these settings. The best was (32, 16) units, dropout 0.4, learning rate 0.01.

6.4 Feature analysis

Permutation importance was computed on the test set (the drop in AUC when a feature is shuffled, averaged over 10 repeats). Tenure dominates (AUC drop 0.065, about 4.5 times any other feature), followed by two-year contract, month-to-month contract and DSL internet service. This agrees with the EDA and statistical tests, so the model has learned the same drivers that the analysis found.

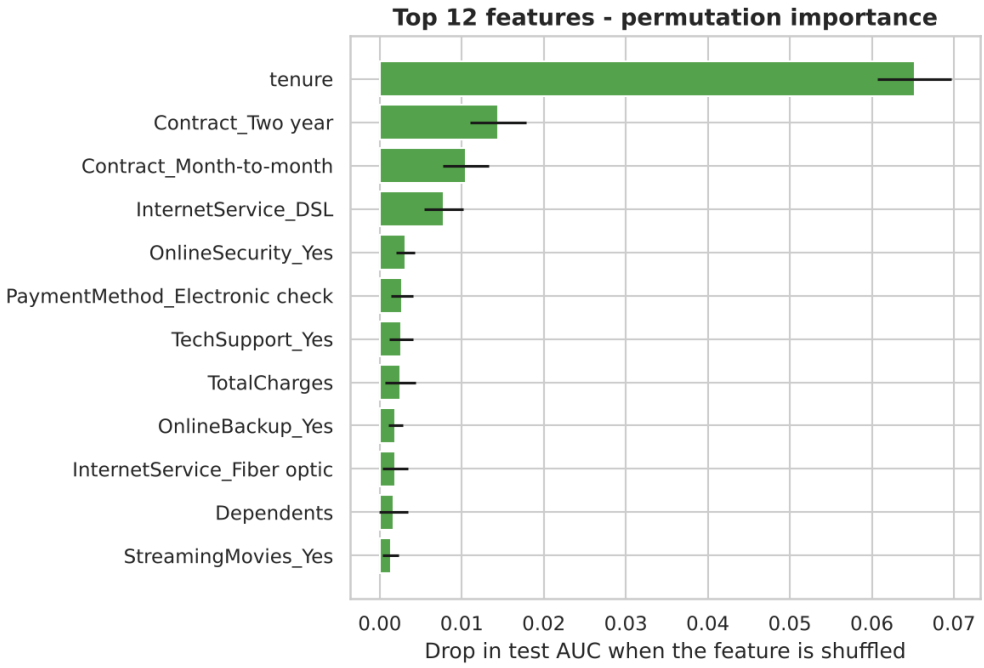


Figure 11: Permutation feature importance (top 12)

7. Result Visualization and Interpretation

Model (test set)	AUC	Accuracy	Precision	Recall	F1
MLP (tuned)	0.839	0.746	0.516	0.762	0.615
Logistic Regression (baseline)	0.845	0.747	-	0.779	0.621

7.1 Loss vs epoch and accuracy vs epoch

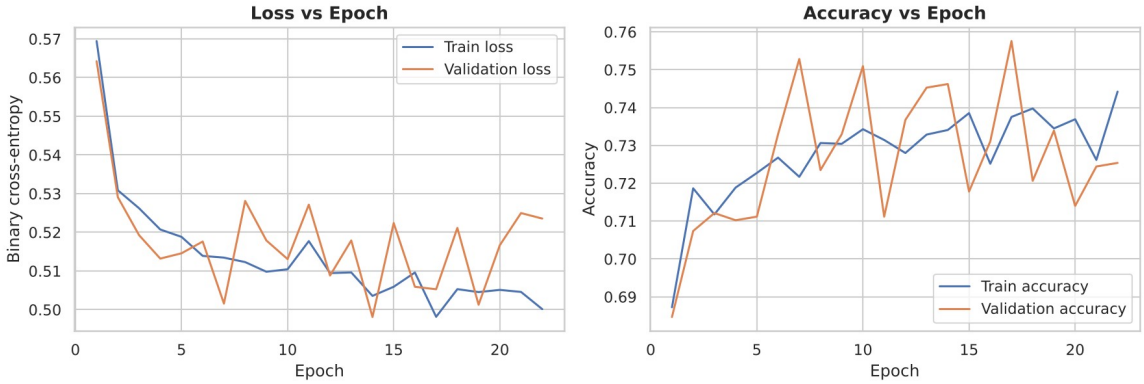


Figure 12: Training and validation loss and accuracy

Training and validation loss fall together within the first five epochs and then flatten at about 0.50-0.52. The two curves stay close, so there is no serious overfitting. The validation curve is noisy because the validation set is small (1,056 rows); early stopping restored the best epoch.

7.2 Confusion matrix

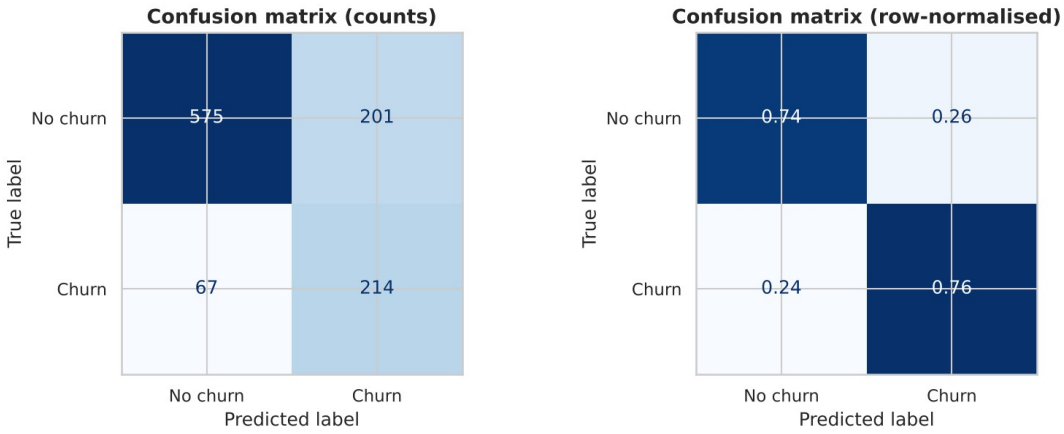


Figure 13: Confusion matrix on the test set

Of 281 real churners the model identifies 214 (recall 76%) and misses 67. Of 776 loyal customers it correctly clears 575 and wrongly flags 201, giving a churn precision of 52%. For a retention campaign, missing a churner is usually costlier than offering a discount to a loyal customer, so the high recall is a useful trade-off.

7.3 ROC curve and AUC

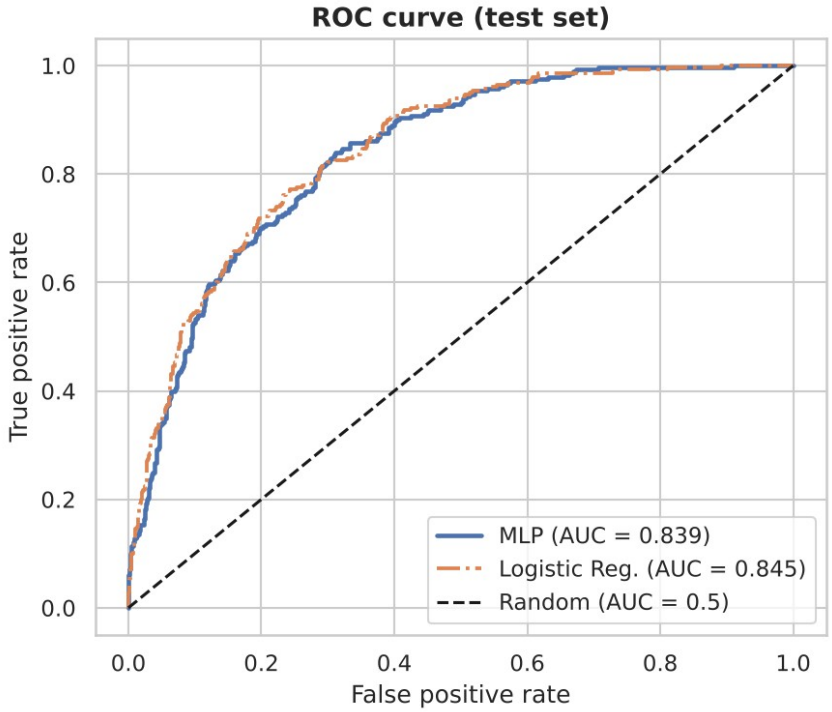


Figure 14: ROC curve of the MLP vs the logistic regression baseline

The AUC is 0.839 (0.5 = random guessing, 1.0 = perfect): a randomly chosen churner receives a higher risk score than a randomly chosen loyal customer 84% of the time. The logistic regression baseline (0.845) is practically identical; the two curves overlap almost everywhere.

7.4 Probability distribution and threshold analysis

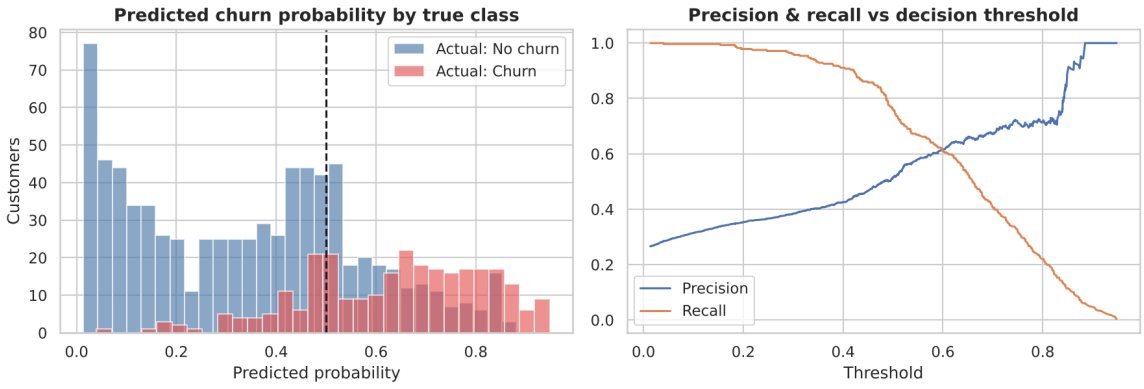


Figure 15: Predicted probability by true class, and precision/recall vs threshold

Churners receive mostly high predicted probabilities and loyal customers low ones, with an overlap zone around 0.4-0.6. The right-hand chart shows the business can trade precision for recall by moving the decision threshold away from 0.5 depending on the cost of a retention offer.

8. Conclusion and Future Scope

8.1 What was learned and key findings

- About 26.5% of customers churn. The riskiest customers are new (low tenure), on month-to-month contracts, using fibre-optic internet, paying by electronic check, without Online Security or Tech Support, and paying higher monthly bills.
- Customers on two-year contracts (2.8% churn), with long tenure (above 60 months: 6.6%) or with many add-on services are very loyal.
- Gender and having a phone line have no significant effect on churn.
- The MLP reached an AUC of 0.839, accuracy of 74.6% and recall of 76.2% on unseen data; it performs the same as logistic regression, which shows the relationships in this data are mostly simple, so a deeper network gives no extra benefit.

8.2 Business recommendations

- Offer discounts or incentives to move month-to-month customers to one- or two-year contracts.
- Pay special attention to customers in their first six months (52.9% of them churn).
- Bundle Online Security and Tech Support with fibre plans and encourage automatic payment instead of electronic check.

8.3 Challenges

Class imbalance, a hidden data issue (blank TotalCharges strings), strongly correlated features, and noisy validation curves due to the limited data size.

8.4 Future scope

- Compare with gradient boosting models (XGBoost / LightGBM) and try SMOTE or focal loss for the imbalance.
- Choose the decision threshold using the real cost of losing a customer versus the cost of a retention offer.
- Add behavioural data such as call-centre complaints, usage trends and billing disputes; use SHAP for per-customer explanations.
- Deploy the model as a REST API or dashboard so the retention team can score customers regularly.

References

[1] "Telco Customer Churn," IBM sample dataset, Kaggle. [Online]. Available: https://www.kaggle.com/datasets/blastchar/telco-customer-churn

[2] J. W. Tukey, Exploratory Data Analysis. Reading, MA, USA: Addison-Wesley, 1977.

[3] B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA, USA: O'Reilly Media, 2007.

[4] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015. arXiv:1412.6980.

[7] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90-95, 2007.

[8] F. Chollet et al., "Keras," 2015. [Online]. Available: https://keras.io

Appendix: Code

The complete, commented code (data loading, EDA, preprocessing, model creation and training, evaluation and visualization) is also available as churn_analysis.ipynb in the GitHub repository.

```
# ---------- 0. Imports & settings ----------
import os, json, warnings, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import plotly.express as px

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, roc_curve, roc_auc_score,
                             accuracy_score, precision_score, recall_score, f1_score,
                             classification_report, precision_recall_curve)
from sklearn.utils.class_weight import compute_class_weight

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

warnings.filterwarnings("ignore")
SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams.update({"figure.dpi": 110, "axes.titlesize": 13, "axes.titleweight": "bold"})
FIG_DIR = "figures"; os.makedirs(FIG_DIR, exist_ok=True)
RESULTS = {}          # numbers collected here are reused in the written report

def savefig(name):
    plt.tight_layout(); plt.savefig(f"{FIG_DIR}/{name}.png", dpi=150, bbox_inches="tight"); plt.show()


# ---------- 1. Data loading (Stage 1: Acquire, Stage 2: Parse) ----------
df = pd.read_csv("data/WA_Fn-UseC_-Telco-Customer-Churn.csv")
print("Shape:", df.shape)
display(df.head())
df.info()


# ---------- 2. Data quality checks (Stage 3: Filter) ----------
print("Duplicate rows (all columns):", df.duplicated().sum())
print("Duplicate customerIDs     :", df["customerID"].duplicated().sum())

# TotalCharges is stored as text because some cells are blank spaces
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
print("\nMissing values after numeric conversion:")
print(df.isna().sum()[df.isna().sum() > 0])
print("\nRows with missing TotalCharges (check tenure):")
display(df[df["TotalCharges"].isna()][["customerID", "tenure", "MonthlyCharges", "TotalCharges",
"Churn"]])

# These customers have tenure = 0 (brand new, not billed yet) -> TotalCharges = 0 is the logical value
df.loc[df["TotalCharges"].isna(), "TotalCharges"] = 0.0

# SeniorCitizen is 0/1 -> make it a readable category like the other Yes/No columns
df["SeniorCitizen"] = df["SeniorCitizen"].map({0: "No", 1: "Yes"})
RESULTS["n_rows"], RESULTS["n_cols"] = int(df.shape[0]), int(df.shape[1])
RESULTS["churn_rate"] = float((df["Churn"] == "Yes").mean())


# ---------- 3. Basic statistics ----------
num_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
cat_cols = [c for c in df.columns if c not in num_cols + ["customerID", "Churn"]]
```

```
print(f"{len(num_cols)} numeric features, {len(cat_cols)} categorical features (+ customerID, Churn
target)")
desc = df[num_cols].describe().T
desc["skewness"] = df[num_cols].skew()
desc["kurtosis"] = df[num_cols].kurt()
display(desc.round(2))
RESULTS["describe"] = desc.round(2).to_dict()
print("\nTarget distribution:")
print(df["Churn"].value_counts(), "\n", df["Churn"].value_counts(normalize=True).round(3))


# ---------- 4. Outlier detection (IQR rule) ----------
out_rows = []
for c in num_cols:
    q1, q3 = df[c].quantile([.25, .75]); iqr = q3 - q1
    lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_out = int(((df[c] < lo) | (df[c] > hi)).sum())
    out_rows.append({"feature": c, "lower_fence": round(lo, 2), "upper_fence": round(hi, 2),
"n_outliers": n_out})
outliers = pd.DataFrame(out_rows); display(outliers)
RESULTS["outliers"] = outliers.to_dict("records")
# Decision: all values are valid business values (no negative/impossible numbers), so we KEEP them.


# ---------- VISUAL 1: Target distribution (Bernoulli) + categorical churn rates ----------
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
cnt = df["Churn"].value_counts()
ax[0].bar(cnt.index, cnt.values, color=["#4C78A8", "#E45756"])
for i, v in enumerate(cnt.values): ax[0].text(i, v + 60, f"{v} ({v/len(df):.1%})", ha="center")
ax[0].set(title="Churn class distribution", ylabel="Customers", ylim=(0, cnt.max() * 1.15))
order = ["Month-to-month", "One year", "Two year"]
rate = df.groupby("Contract")["Churn"].apply(lambda s: (s == "Yes").mean()).reindex(order)
sns.barplot(x=rate.index, y=rate.values * 100, ax=ax[1], palette="Reds")
for i, v in enumerate(rate.values): ax[1].text(i, v * 100 + 1, f"{v:.1%}", ha="center")
ax[1].set(title="Churn rate by contract type", ylabel="Churn rate (%)", xlabel="", ylim=(0, rate.max()
* 100 * 1.15))
savefig("01_target_and_contract")
RESULTS["churn_by_contract"] = rate.round(4).to_dict()


# ---------- VISUAL 2: Histograms + KDE of numeric features split by churn ----------
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
for a, c in zip(ax, num_cols):
    sns.histplot(data=df, x=c, hue="Churn", kde=True, stat="density", common_norm=False,
                 palette={"No": "#4C78A8", "Yes": "#E45756"}, ax=a, bins=30)
    a.set(title=f"Distribution of {c}", xlabel=c)
savefig("02_numeric_histograms")
for c in num_cols:
    print(c, "| mean churn=Yes: %.1f | mean churn=No: %.1f" % (df.loc[df.Churn=="Yes", c].mean(),
df.loc[df.Churn=="No", c].mean()))
RESULTS["means_by_churn"] = {c: {"Yes": float(df.loc[df.Churn=="Yes", c].mean()), "No":
float(df.loc[df.Churn=="No", c].mean())} for c in num_cols}


# ---------- VISUAL 3: Boxplots of numeric features by churn (outlier view) ----------
fig, ax = plt.subplots(1, 3, figsize=(14, 4))
for a, c in zip(ax, num_cols):
    sns.boxplot(data=df, x="Churn", y=c, palette={"No": "#4C78A8", "Yes": "#E45756"}, ax=a)
    a.set(title=f"{c} by Churn")
savefig("03_boxplots")


# ---------- VISUAL 4: Correlation heatmap (numeric + encoded target) ----------
tmp = df[num_cols].copy(); tmp["Churn (1=Yes)"] = (df["Churn"] == "Yes").astype(int)
tmp["Services_count"] =
(df[["OnlineSecurity","OnlineBackup","DeviceProtection","TechSupport","StreamingTV","StreamingMovies"]]
== "Yes").sum(axis=1)
corr = tmp.corr()
plt.figure(figsize=(7, 5.5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0, vmin=-1, vmax=1, linewidths=.5)
plt.title("Correlation heatmap (Pearson)")
savefig("04_correlation_heatmap")
display(corr["Churn (1=Yes)"].round(3))
RESULTS["corr_with_churn"] = corr["Churn (1=Yes)"].round(3).to_dict()
RESULTS["corr_tenure_total"] = float(corr.loc["tenure", "TotalCharges"])
```

```
# ---------- VISUAL 5: Scatter plot - MonthlyCharges vs tenure coloured by churn ----------
plt.figure(figsize=(8, 5.5))
sns.scatterplot(data=df, x="tenure", y="MonthlyCharges", hue="Churn", alpha=.45, s=22,
                palette={"No": "#4C78A8", "Yes": "#E45756"})
plt.title("Monthly charges vs tenure"); plt.xlabel("Tenure (months)"); plt.ylabel("Monthly charges
($)")
savefig("05_scatter_tenure_monthly")


# ---------- VISUAL 6: Pairplot ----------
pp = sns.pairplot(df[num_cols + ["Churn"]], hue="Churn", palette={"No": "#4C78A8", "Yes": "#E45756"},
                  corner=True, plot_kws={"alpha": .35, "s": 14}, height=2.4)
pp.fig.suptitle("Pairplot of numeric features", y=1.02, fontweight="bold")
pp.savefig(f"{FIG_DIR}/06_pairplot.png", dpi=150, bbox_inches="tight"); plt.show()


# ---------- VISUAL 7: Churn rate by every categorical feature (ranked) ----------
rows = []
for c in cat_cols:
    for lvl, s in df.groupby(c)["Churn"]:
        rows.append({"feature": c, "level": lvl, "label": f"{c} = {lvl}", "churn_rate": (s ==
"Yes").mean(), "n": len(s)})
cr = pd.DataFrame(rows).sort_values("churn_rate")
plt.figure(figsize=(8, 9))
colors = ["#E45756" if v > RESULTS["churn_rate"] else "#4C78A8" for v in cr["churn_rate"]]
plt.barh(cr["label"], cr["churn_rate"] * 100, color=colors)
plt.axvline(RESULTS["churn_rate"] * 100, color="k", ls="--", lw=1, label=f"Overall churn
{RESULTS['churn_rate']:.1%}")
plt.xlabel("Churn rate (%)"); plt.title("Churn rate for each category level"); plt.legend(loc="lower
right")
plt.yticks(fontsize=8)
savefig("07_churn_rate_by_category")
RESULTS["top_levels"] = cr.tail(5)[["label", "churn_rate"]].round(3).to_dict("records")
RESULTS["bottom_levels"] = cr.head(5)[["label", "churn_rate"]].round(3).to_dict("records")


# ---------- VISUAL 8 (Stage 7: Interact): interactive Plotly chart, saved as HTML ----------
fig = px.scatter(df, x="tenure", y="MonthlyCharges", color="Churn", symbol="Contract", opacity=.6,
                 hover_data=["InternetService", "PaymentMethod", "TotalCharges"],
                 color_discrete_map={"No": "#4C78A8", "Yes": "#E45756"},
                 title="Interactive: tenure vs monthly charges (hover, zoom, filter by legend)")
fig.write_html(f"{FIG_DIR}/08_interactive_scatter.html")
fig.show()


# ---------- 5. Univariate / distribution analysis (statistical) ----------
# Continuous: normality tests, skewness, Q-Q plots, fitted normal curve
fig, ax = plt.subplots(2, 3, figsize=(15, 8))
norm_rows = []
for j, c in enumerate(num_cols):
    x = df[c]
    mu, sd = x.mean(), x.std()
    sns.histplot(x, stat="density", bins=30, color="#9ecae1", ax=ax[0, j])
    xs = np.linspace(x.min(), x.max(), 300)
    ax[0, j].plot(xs, stats.norm.pdf(xs, mu, sd), "r-", lw=2, label="Fitted Normal")
    ax[0, j].set(title=f"{c}: histogram vs fitted Normal"); ax[0, j].legend()
    stats.probplot(x, dist="norm", plot=ax[1, j]); ax[1, j].set_title(f"{c}: Q-Q plot")
    k2, p = stats.normaltest(x)
    norm_rows.append({"feature": c, "skew": round(x.skew(), 3), "excess_kurtosis": round(x.kurt(), 3),
                      "D'Agostino K2": round(k2, 1), "p_value": float(p), "normal?": "No" if p < .05
else "Yes"})
savefig("09_continuous_distributions")
norm_df = pd.DataFrame(norm_rows); display(norm_df)
RESULTS["normality"] = norm_df.to_dict("records")


# ---------- 6. Discrete probability distributions ----------
# (a) Churn ~ Bernoulli(p)   (b) number of add-on services (0-6) vs Binomial(6, p)
p_churn = RESULTS["churn_rate"]
print(f"Churn ~ Bernoulli(p = {p_churn:.3f});  Var = p(1-p) = {p_churn*(1-p_churn):.3f}")
addon =
["OnlineSecurity","OnlineBackup","DeviceProtection","TechSupport","StreamingTV","StreamingMovies"]
n_serv = (df[addon] == "Yes").sum(axis=1)
obs = n_serv.value_counts(normalize=True).sort_index()
p_hat = n_serv.mean() / 6
```

```
k = np.arange(0, 7); binom_pmf = stats.binom.pmf(k, 6, p_hat)
fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
ax[0].bar(k - .2, obs.reindex(k, fill_value=0).values, width=.4, label="Observed", color="#4C78A8")
ax[0].bar(k + .2, binom_pmf, width=.4, label=f"Binomial(6, {p_hat:.2f})", color="#F58518")
ax[0].set(title="Number of add-on services per customer", xlabel="Add-on services subscribed",
ylabel="Probability"); ax[0].legend()
chr_ = df.groupby(n_serv)["Churn"].apply(lambda s: (s == "Yes").mean())
ax[1].plot(chr_.index, chr_.values * 100, "o-", color="#E45756")
ax[1].set(title="Churn rate vs number of add-on services", xlabel="Add-on services subscribed",
ylabel="Churn rate (%)")
savefig("10_discrete_distributions")
# chi-square goodness-of-fit (merge sparse tail so expected counts >= 5)
exp = binom_pmf * len(df); obs_c = n_serv.value_counts().reindex(k, fill_value=0).values
chi2, pv = stats.chisquare(obs_c, exp * obs_c.sum() / exp.sum())
print(f"Chi-square GOF vs Binomial: chi2={chi2:.1f}, p={pv:.2e}  -> {'rejects' if pv<.05 else 'does not
reject'} Binomial")
RESULTS["binom"] = {"p_hat": float(p_hat), "chi2": float(chi2), "p": float(pv)}
RESULTS["churn_by_nservices"] = chr_.round(3).to_dict()


# ---------- 7. Bivariate / multivariate statistical tests ----------
# Numeric vs Churn: Mann-Whitney U (non-normal data) ; Categorical vs Churn: chi-square + Cramer's V
tests = []
for c in num_cols:
    u, p = stats.mannwhitneyu(df.loc[df.Churn == "Yes", c], df.loc[df.Churn == "No", c])
    tests.append({"feature": c, "test": "Mann-Whitney U", "statistic": round(u, 0), "p_value": p,
"effect": round(float(stats.pointbiserialr((df.Churn=='Yes').astype(int), df[c])[0]), 3)})
for c in cat_cols:
    ct = pd.crosstab(df[c], df["Churn"]); chi2, p, dof, _ = stats.chi2_contingency(ct)
    v = np.sqrt(chi2 / (ct.values.sum() * (min(ct.shape) - 1)))
    tests.append({"feature": c, "test": "Chi-square", "statistic": round(chi2, 1), "p_value": p,
"effect": round(v, 3)})
tests = pd.DataFrame(tests).sort_values("effect", key=abs, ascending=False)
tests["significant"] = tests["p_value"] < .05
display(tests.assign(p_value=tests.p_value.map("{:.2e}".format)).reset_index(drop=True))
RESULTS["tests_top"] =
tests.head(8).assign(p_value=tests.head(8).p_value.map("{:.2e}".format)).to_dict("records")
RESULTS["not_significant"] = tests.loc[~tests.significant, "feature"].tolist()

plt.figure(figsize=(7.5, 6.5))
tt = tests.sort_values("effect", key=abs)
plt.barh(tt["feature"], tt["effect"].abs(), color=["#E45756" if s else "#BBBBBB" for s in
tt["significant"]])
plt.xlabel("Effect size (|Cramer's V| for categorical, |point-biserial r| for numeric)")
plt.title("Association of each feature with churn (red = significant at 5%)")
savefig("11_association_strength")


# ---------- 8. Preprocessing: encoding, split, scaling ----------
data = df.drop(columns=["customerID"]).copy()
y = (data.pop("Churn") == "Yes").astype(int).values
# Binary Yes/No-style columns -> 0/1 ; multi-class columns -> one-hot
bin_cols = [c for c in cat_cols if data[c].nunique() == 2]
multi_cols = [c for c in cat_cols if data[c].nunique() > 2]
for c in bin_cols:
    vals = sorted(data[c].unique())
    pos = "Yes" if "Yes" in vals else ("Female" if "Female" in vals else vals[-1])
    data[c] = (data[c] == pos).astype(int)
X = pd.get_dummies(data, columns=multi_cols, drop_first=False).astype(float)
print("Binary-encoded:", bin_cols); print("One-hot encoded:", multi_cols)
print("Final feature matrix:", X.shape)

# 70 / 15 / 15 stratified split
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=.30, stratify=y, random_state=SEED)
X_va, X_te, y_va, y_te = train_test_split(X_tmp, y_tmp, test_size=.50, stratify=y_tmp,
random_state=SEED)
# Standardise numeric columns using ONLY training statistics (avoids data leakage)
scaler = StandardScaler().fit(X_tr[num_cols])
for part in (X_tr, X_va, X_te): part.loc[:, num_cols] = scaler.transform(part[num_cols])
feature_names = list(X.columns)
print(f"Train {X_tr.shape} | Validation {X_va.shape} | Test {X_te.shape}")
print("Churn rate  train/val/test: %.3f / %.3f / %.3f" % (y_tr.mean(), y_va.mean(), y_te.mean()))
RESULTS["split"] = {"train": len(X_tr), "val": len(X_va), "test": len(X_te), "n_features": X.shape[1]}
X_tr, X_va, X_te = X_tr.values.astype("float32"), X_va.values.astype("float32"),
X_te.values.astype("float32")
```

```
# ---------- 9. Baseline model (Logistic Regression) for comparison ----------
lr = LogisticRegression(max_iter=2000, class_weight="balanced").fit(X_tr, y_tr)
lr_prob = lr.predict_proba(X_te)[:, 1]
RESULTS["baseline"] = {"auc": float(roc_auc_score(y_te, lr_prob)), "acc": float(accuracy_score(y_te,
lr_prob > .5)),
                       "recall": float(recall_score(y_te, lr_prob > .5)), "f1": float(f1_score(y_te,
lr_prob > .5))}
print("Logistic Regression baseline:", {k: round(v, 3) for k, v in RESULTS["baseline"].items()})


# ---------- 10. Deep Learning model: MLP + basic hyper-parameter tuning ----------
cw = compute_class_weight("balanced", classes=np.array([0, 1]), y=y_tr)
class_weight = {0: cw[0], 1: cw[1]}
print("Class weights:", {k: round(v, 2) for k, v in class_weight.items()})

def build_mlp(input_dim, units=(64, 32), dropout=0.3, lr=1e-3):
    m = keras.Sequential([layers.Input(shape=(input_dim,))])
    for u in units:
        m.add(layers.Dense(u, activation="relu", kernel_initializer="he_normal"))
        m.add(layers.Dropout(dropout))
    m.add(layers.Dense(1, activation="sigmoid"))
    m.compile(optimizer=keras.optimizers.Adam(lr), loss="binary_crossentropy",
              metrics=["accuracy", keras.metrics.AUC(name="auc")])
    return m

def fit_model(units, dropout, lr, epochs=100, verbose=0):
    keras.utils.set_random_seed(SEED)
    m = build_mlp(X_tr.shape[1], units, dropout, lr)
    es = keras.callbacks.EarlyStopping(monitor="val_auc", mode="max", patience=12,
restore_best_weights=True)
    h = m.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=epochs, batch_size=64,
              class_weight=class_weight, callbacks=[es], verbose=verbose)
    return m, h

grid = [(u, d, l) for u in [(32, 16), (64, 32), (128, 64)] for d in [0.2, 0.4] for l in [1e-3, 1e-2]]
tune_rows = []
for units, drop, lr_ in grid:
    m, h = fit_model(units, drop, lr_)
    va_auc = roc_auc_score(y_va, m.predict(X_va, verbose=0).ravel())
    tune_rows.append({"hidden_units": str(units), "dropout": drop, "learning_rate": lr_,
                      "epochs_run": len(h.history["loss"]), "val_auc": round(va_auc, 4)})
    print(tune_rows[-1])
tune = pd.DataFrame(tune_rows).sort_values("val_auc", ascending=False).reset_index(drop=True)
display(tune)
best = grid[int(pd.DataFrame(tune_rows)["val_auc"].idxmax())]
print("\nBest config (by validation AUC):", best)
RESULTS["tuning"] = tune.to_dict("records"); RESULTS["best_cfg"] = {"units": list(best[0]), "dropout":
best[1], "lr": best[2]}


# ---------- 11. Train final model with the best hyper-parameters ----------
model, hist = fit_model(*best, verbose=0)
model.summary()
RESULTS["epochs_trained"] = len(hist.history["loss"])
RESULTS["n_params"] = int(model.count_params())


# ---------- 12. Evaluation on the held-out TEST set ----------
prob = model.predict(X_te, verbose=0).ravel()
pred = (prob >= 0.5).astype(int)
metrics = {"auc": roc_auc_score(y_te, prob), "acc": accuracy_score(y_te, pred), "precision":
precision_score(y_te, pred),
           "recall": recall_score(y_te, pred), "f1": f1_score(y_te, pred)}
RESULTS["mlp"] = {k: float(v) for k, v in metrics.items()}
print("MLP test metrics:", {k: round(v, 3) for k, v in metrics.items()})
print(classification_report(y_te, pred, target_names=["No churn", "Churn"]))
cmp = pd.DataFrame({"Logistic Regression": RESULTS["baseline"], "MLP (tuned)":
RESULTS["mlp"]}).T[["auc", "acc", "recall", "f1"]].round(3)
display(cmp)

# Chart A+B: loss & accuracy vs epoch
ep = range(1, len(hist.history["loss"]) + 1)
fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
ax[0].plot(ep, hist.history["loss"], label="Train loss"); ax[0].plot(ep, hist.history["val_loss"],
label="Validation loss")
```

```
ax[0].set(title="Loss vs Epoch", xlabel="Epoch", ylabel="Binary cross-entropy"); ax[0].legend()
ax[1].plot(ep, hist.history["accuracy"], label="Train accuracy"); ax[1].plot(ep,
hist.history["val_accuracy"], label="Validation accuracy")
ax[1].set(title="Accuracy vs Epoch", xlabel="Epoch", ylabel="Accuracy"); ax[1].legend()
savefig("12_loss_accuracy_vs_epoch")

# Chart C: confusion matrix
cm = confusion_matrix(y_te, pred); RESULTS["cm"] = cm.tolist()
fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
ConfusionMatrixDisplay(cm, display_labels=["No churn", "Churn"]).plot(ax=ax[0], cmap="Blues",
colorbar=False)
ax[0].set_title("Confusion matrix (counts)")
ConfusionMatrixDisplay(confusion_matrix(y_te, pred, normalize="true"), display_labels=["No churn",
"Churn"]).plot(ax=ax[1], cmap="Blues", colorbar=False, values_format=".2f")
ax[1].set_title("Confusion matrix (row-normalised)")
savefig("13_confusion_matrix")

# Chart D: ROC curve (MLP vs baseline)
fpr, tpr, _ = roc_curve(y_te, prob); fpr2, tpr2, _ = roc_curve(y_te, lr_prob)
plt.figure(figsize=(6, 5.2))
plt.plot(fpr, tpr, lw=2.2, label=f"MLP (AUC = {metrics['auc']:.3f})")
plt.plot(fpr2, tpr2, lw=1.8, ls="-.", label=f"Logistic Reg. (AUC = {RESULTS['baseline']['auc']:.3f})")
plt.plot([0, 1], [0, 1], "k--", label="Random (AUC = 0.5)")
plt.xlabel("False positive rate"); plt.ylabel("True positive rate"); plt.title("ROC curve (test set)");
plt.legend(loc="lower right")
savefig("14_roc_curve")

# Chart E: predicted probability distribution + precision/recall vs threshold
fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
ax[0].hist(prob[y_te == 0], bins=30, alpha=.65, label="Actual: No churn", color="#4C78A8")
ax[0].hist(prob[y_te == 1], bins=30, alpha=.65, label="Actual: Churn", color="#E45756")
ax[0].axvline(.5, color="k", ls="--"); ax[0].set(title="Predicted churn probability by true class",
xlabel="Predicted probability", ylabel="Customers"); ax[0].legend()
pr, rc, th = precision_recall_curve(y_te, prob)
ax[1].plot(th, pr[:-1], label="Precision"); ax[1].plot(th, rc[:-1], label="Recall")
ax[1].set(title="Precision & recall vs decision threshold", xlabel="Threshold"); ax[1].legend()
savefig("15_probability_and_threshold")


# ---------- 13. Feature importance (permutation importance on the test set, measured by AUC drop)
----------
rng = np.random.RandomState(SEED); base_auc = metrics["auc"]; imp = []
for j, name in enumerate(feature_names):
    drops = []
    for _ in range(10):
        Xp = X_te.copy(); Xp[:, j] = rng.permutation(Xp[:, j])
        drops.append(base_auc - roc_auc_score(y_te, model.predict(Xp, verbose=0,
batch_size=2048).ravel()))
    imp.append((name, np.mean(drops), np.std(drops)))
imp = pd.DataFrame(imp, columns=["feature", "auc_drop", "std"]).sort_values("auc_drop",
ascending=False)
top = imp.head(12).iloc[::-1]
plt.figure(figsize=(8, 5.5))
plt.barh(top["feature"], top["auc_drop"], xerr=top["std"], color="#54A24B")
plt.xlabel("Drop in test AUC when the feature is shuffled"); plt.title("Top 12 features - permutation
importance")
savefig("16_feature_importance")
display(imp.head(10).round(4))
RESULTS["importance_top"] = imp.head(8).round(4)[["feature", "auc_drop"]].to_dict("records")


# ---------- 14. Save model & results ----------
os.makedirs("model", exist_ok=True)
model.save("model/churn_mlp.keras")
json.dump(RESULTS, open("results.json", "w"), indent=2, default=float)
print("Saved model/churn_mlp.keras and results.json")
```